ChatGPT

StreamerSonglist MCP Enhancement Guide

Overview

The StreamerSonglist MCP server acts as an intermediary between your Twitch bot and the StreamerSonglist API. To build richer bot interactions—like finding similar songs, managing song attributes, or leveraging new API features—you should extend MCP to call a broader set of endpoints and perform local processing before returning results to chat. Below are concrete enhancements based on the official v1 API schema [1] [2].

1. Expand API Client Coverage

The existing MCP code primarily interacts with queue and basic song endpoints. Add client methods for the following:

- **Song list operations:** implement `getSongs(streamerId)`, `createSong(streamerId, songData)`, `updateSongs(streamerId, songs)` and `deleteSongs(streamerId, ids)` against `/v1/streamers/{streamerId}/songs` [3]. Include pagination if supported and error handling for 4xx/5xx statuses.
- **Song export/import:** implement `exportSongs(streamerId)` and `importSongs(streamerId, file)` using `/v1/streamers/{streamerId}/songs/export` and `/v1/streamers/{streamerId}/songs/song-import` [4] [5].
- **Song validation:** expose `/v1/streamers/{streamerId}/songs/song-validation` to let moderators validate a title/artist pair before adding it [6].
- **Queue enhancements:** wrap endpoints for marking songs played, moving entries to saved queue, checking request eligibility and saved queue management [2] [7].
- **User and auth endpoints:** implement optional flows for bots or user login via `/v1/auth/bot` and Twitch/Streamlabs OAuth endpoints [8].
- **Feature flags and reward endpoints:** add administrative commands to toggle feature flags or manage channel point rewards [9] [10].

By surfacing these routes in the MCP client layer, the bot will be able to issue richer commands and remain aligned with API updates.

2. Implement Similar Song Search

Users often mistype song titles or look for recommendations. There isn't a dedicated "similar songs" endpoint, so implement this logic in MCP:

1. **Fetch the full song list** for the streamer via `getSongs(streamerId)`.
2. **Perform fuzzy matching** locally. Use a library like `rapidfuzz` (or Python's `difflib.get_close_matches`) to rank songs by Levenshtein distance to the user's query.

3. **Return top matches** to the bot for display. Include artist/title and optional attributes.

Example pseudo-code:

```
from rapidfuzz import process

def find_similar_songs(streamer_id, query, limit=5):
    songs = api_client.getSongs(streamer_id)  # list of {"title", "artist", ...}
    candidates = [f"{s['title']} - {s['artist']}" for s in songs]
    matches = process.extract(query, candidates, limit=limit,
scorer=process.fuzz.ratio)
    return matches
```

Add a bot command like `!similar <query>` that calls this function and returns the top results.

3. Add Attribute-Based Filtering

Song attributes can tag tracks (e.g., “ballad,” “hype,” “new”). Use `/v1/streamers/{streamerId}/songAttributes` to list attributes and `/v1/streamers/{streamerId}/songAttributes/{attributeId}/activate-songs` or `deactivate-songs` to manage song-attribute associations [11] [12]. Extend MCP to:

- **List attributes:** display available tags in chat or a GUI.
- **Filter songs by attribute:** after fetching songs, filter by a chosen `attributeId`.
- **Toggle attributes on songs:** for moderators, add commands to activate/deactivate attributes on a song ID.

These features allow your bot to narrow suggestions (e.g., “show me hype songs similar to ‘Bohemian Rhapsody’”).

4. Improve Error Handling and Rate Limiting

The API returns various HTTP codes (200, 201, 204, 400+, 503). Wrap calls with retries and exponential backoff. Respect `429 Too Many Requests` by queuing retries and informing the user when the service is busy. For long-running operations (song import/export), consider asynchronous tasks and update the bot when complete.

5. Caching and Pagination

To reduce latency and API usage, cache recent results (e.g., song lists, attributes) in memory or a lightweight datastore. Invalidate caches on writes (POST/PUT/DELETE) or after a TTL. Implement pagination support if the API adds query parameters (e.g., `page` or `limit`), so the MCP can handle large song libraries.

6. Modularity and Testing

Organize API calls into a dedicated service class (e.g., `StreamersonglistClient`) with typed methods. Write unit tests using mocked responses to ensure the MCP continues working as the API evolves. For integration tests, set up a sandbox StreamerSonglist server or use the `cypress` auth endpoint for automated workflows [13].

7. Default Streamer Handling & Safe Mode

The existing MCP server supports a `DEFAULT_STREAMER` environment variable to simplify tool calls: if a user omits `streamerName` when invoking `getQueue`, `getQueueStats` or `getStreamerByName`, the server automatically uses the configured default [14]. When enhancing the server, keep this behaviour for convenience but ensure it's explicit and overridable:

- **Expose a configuration interface** (e.g., command line flag or GUI) to set or update `DEFAULT_STREAMER` without editing environment variables.
- **Validate default exists**: on startup, call `getStreamerByName` with the default; if it returns `404`, prompt for a valid name rather than failing silently.
- **Allow overrides**: even when `DEFAULT_STREAMER` is set, your enhanced tools (e.g., `searchSongs` or `findSimilarSongs`) should accept a `streamerName` parameter to override the default.

The branch `codex/update-default-streamer-handling` intentionally limits the server to **public, read-only GET endpoints** for safety [15]. When adding write operations (POST/PUT/DELETE), consider creating a separate "privileged" mode that requires an API token and is disabled by default. Document these differences clearly so users understand the risks and can opt in when necessary.

Conclusion

By expanding API coverage, adding local fuzzy matching for similar songs, supporting attribute-based filtering, and improving robustness, your MCP server will provide a richer experience for both streamers and viewers. These enhancements leverage the full StreamerSonglist v1 API and position your bot to grow alongside future API features.

---

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] api.streamersonglist.com
https://api.streamersonglist.com/docs/swagger-ui-init.js

[14] [15] raw.githubusercontent.com
https://raw.githubusercontent.com/vuvuvu/streamersonglist-mcp/codex/update-default-streamer-handling/README.md